

Cryptographic Design Document

CS4455 Cybersecurity Epic Project 2026

Team kfc — Epic Messaging

Module: Cryptography (Eoin O'Brien)

Epic Messaging is a secure relay messaging system. Message bodies are end-to-end encrypted before they reach the server. Passwords and long-term private keys are protected with standard KDFs and AEAD. The Qt client uses OpenSSL for production traffic; the cryptography/ package implements the full Signal-style session for automated tests and backend integration.

1. Security goals

The design targets confidentiality, integrity, and authenticity of message content between users.

Property	Mechanism
Confidentiality of message bodies	AES-256-GCM inside an E2EE session; server stores ciphertext only
Integrity of message bodies	GCM authentication tag; tampering fails decryption
Authenticity of sender	Ed25519-signed pre-keys plus TOFU identity pinning on clients
Password confidentiality if DB leaks	Argon2id PHC hashes; no plaintext passwords
Private keys at rest on client	HKDF-derived keys and AES-256-GCM via <code>encryptPrivateKeyForStorage</code>

TLS protects the transport to the API host. E2EE protects content from the server operator.

2. Threat model

We consider four attacker classes. The tables below separate cryptographic guarantees from properties the protocol does not provide.

Attacker	Capability
A. Passive network	Read client-server traffic
B. Active network	Modify, drop, replay, or inject traffic
C. Honest-but-curious server	Follows the API; logs ciphertext and metadata

D. Compromised server	Full database; can return malicious API responses
-----------------------	---

2.1 Cryptographic defences

Defence	A	B	C	D
Past ciphertext stays confidential	Yes	Yes	Yes	Yes
Ciphertext tampering is detected	n/a	Yes	Yes	Yes
Sender cryptographically bound to keys	n/a	Yes	Yes	Yes*
Forward secrecy for new messages	Yes	Yes	Yes	Yes**

*Clients pin identity keys with /trust and block sends if the server swaps keys.

**Full double ratchet in the TypeScript package; the C++ client runs X3DH plus GCM on each message until persisted ratchet state is added.

2.2 Out of scope

These properties are not provided by the cipher layer or are handled operationally:

Topic	Status	Explanation
Hide who messaged whom	Not provided	Relay server and TLS observer see routing metadata
Guarantee message delivery	Not provided	Server can drop or withhold messages
First-contact MITM	Mitigated by TOFU	User runs /trust before sending

2.3 Compromised server

- With full database access, the server still cannot:

- Decrypt `wire_payload_json` without client private keys.
- Forge AEAD ciphertext that decrypts to chosen plaintext for an existing session.
- The server can observe metadata, delay delivery, or substitute pre-keys for contacts not yet pinned.

3. Protocol construction

3.1 Registration and key publication

1. Client generates X25519 and Ed25519 key pairs and pre-keys.
2. Client registers over TLS; server stores Argon2id password hash only.
3. Client uploads public pre-key bundle; private keys remain on device.
4. Private key blobs may be encrypted at rest under HKDF-derived keys.

3.2 Send and receive

1. Sender fetches recipient public bundle after `/trust` pins identity.
2. TypeScript (cryptography/): X3DH, double ratchet, GCM envelope, libsignal-v1 wire JSON.
3. C++ client: OpenSSL X3DH and GCM per message; each send carries a full X3DH block today.
4. Server stores `wire_payload_json`; recipient decrypts locally.
5. Backend checks JSON shape only; it never decrypts payloads.

3.3 Trust model

Public keys are stored on the server. Clients pin the identity key on first use (`/trust`) and stop sending if the identity changes. We use TOFU rather than a certificate authority.

4. Primitive choices

Libraries: Node crypto, argon2, @privacyresearch/libsignal-protocol-typescript; C++ client: OpenSSL 3 EVP.

4.1 Argon2id

Parameter	Value	Reference
Type	Argon2id	RFC 9106
Memory	64 MiB	OWASP Password Storage Cheat Sheet 2024 (strong tier)
Iterations	3	OWASP 2024
Parallelism	4	OWASP 2024

4.2 HKDF-SHA256 and AES-256-GCM

HKDF (RFC 5869) uses separate info strings for local storage keys versus session keys. AES-256-GCM (NIST SP 800-38D) uses a random 12-byte nonce per encryption. Reusing a nonce under the same key breaks confidentiality; we use random IVs and ratchet-derived message keys.

4.3 X3DH and session keys

HPKE Mode_Auth (RFC 9180) is one standard option for authenticated key establishment. We use X3DH (Signal, 2016) with pre-key bundles on the relay server. The TypeScript package adds the double ratchet (Signal, 2016) for ongoing messages. X25519 (RFC 7748) and Ed25519 (RFC 8032) handle agreement and signed pre-keys.

5. Implementation

Component	Role
cryptography/src/cryptoEngine.ts	Argon2id, HKDF, AES-GCM helpers
cryptography/src/signal/	X3DH, double ratchet, TOFU, libsignal-v1 wire
client NativeSignalCryptoProvider	OpenSSL: keys, pre-key verify, X3DH+GCM per message
FastAPI backend	Relay; Argon2id passwords; no payload decryption

6. Limitations

- Metadata (participants, timing) is visible to the server.
- The C++ client does not yet persist double-ratchet state (see client README).
- C++ and TypeScript inner wire formats are not interoperable without a shared adapter.
- X25519 is not post-quantum; Kyber/PQXDH is reserved for a future libsignal upgrade.
- Users should run /trust before the first message to a new contact.

7. Validation

Check	Command
E2EE round-trip (TypeScript)	cd cryptography && npm run smoke:signal
API relay	npm run e2e:backend
C++ X3DH + GCM	client_tests with OpenSSL enabled

References

- RFC 5869, RFC 7748, RFC 8032, RFC 9106, RFC 9180
- NIST SP 800-38D, NIST SP 800-57
- OWASP Password Storage Cheat Sheet (2024)
- Signal X3DH and Double Ratchet specifications (2016)